12/06/2023

```
program PPL
  var i: integer
      a: array[1..2] of integer;

  procedure f(x: integer);
    begin
      a[1]:=6;
      i := 2;
      x := x+3;
    end;

  begin
    a[1]:= 1;
    a[2]:= 2;
    i := 1;
    f(a[i]);
    write(a[1], a[2], i);
  end;
```

a) Pass by value

6, 2, 2

b) Pass by reference

9, 2, 2

c) Pass by value result

4, 2, 2

d) Pass by name

6, 5, 2 (x → a[i])

```
int x;
void set_x(int n)
  x = n;
void print_x()
  printf("%d\n", x)
void foo(void (S*)(int m), void (P*)(void), int n)
  int x=0
  if((n==1) || (n==3))
    set_x(n)
  else
    S(n)
  if((n==1) || (n==2))
    print_x;
  else
    P()
int main()
  set_x(0); foo(set_x, print_x, 1); print_x()
  set_x(0); foo(set_x, print_x, 2); print_x()
  set_x(0); foo(set_x, print_x, 3); print_x()
  set_x(0); foo(set_x, print_x, 4); print_x()
```

a) Shallow binding

1 0 2 0 3 0 4 0

b) Deep binding

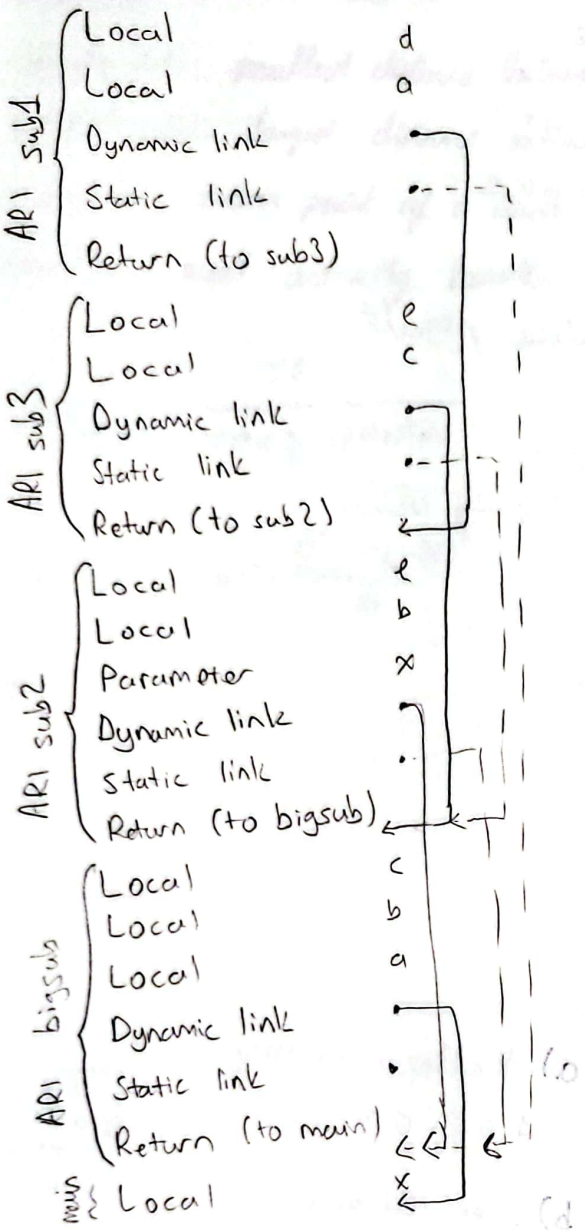
1 0 0 2 0 0 4 4

assume that dynamic scoping is used

Shallow binding: uses the env. of the exp. that invokes the passed function

deep binding: uses the env. at the time the passed function was created

ad hoc binding: uses the env. of the invocation expr. in which the procedure is passed as an argument




```
function main() {
  var x;
  function bigsub() {
    var a, b, c
    function sub1() {
      //var a, d
      ...
      a = b + c
      ...
    }
    function sub2(x) {
      var b, e
      function sub3() {
        var c, e
        ...
        sub1()
        ...
        e = b+a
      }
      ...
      sub3()
      ...
      a = d+e
    }
    ...
    sub2(7)
    ...
  }
  ...
  bigsub()
  ...
}
```

local
parameter
dynamic
static
return

Shallow ARI when deep access method is used to implement dynamic scoping

```
void sub3()
  int x, z;
  x = u+v
void sub2()
  int w, x
void sub1()
  int v, w
void main()
  int v, u
```

main calls sub1
sub1 calls sub1
sub1 calls sub2
sub2 calls sub3

sub3
local
local
dynamic l.
returns to sub2

sub2
local
local
dynamic l.
return to sub1

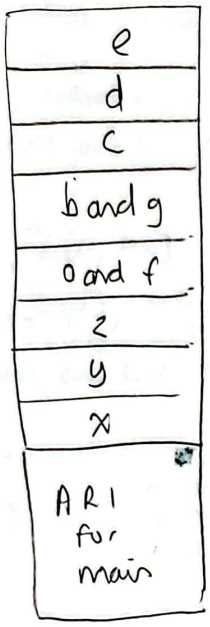
sub1
local
local
dynamic l.
return to sub1

sub1
local
local
dynamic l.
return to main

local

Blocks

```
void main(){
  int x, y, z
  while(...){
    int a, b, c
    ...
    while (...){
      int d, e
      ...
    }
  }
  while (...){
    int f, g
    ...
  }
}
```



Associativity and Precedence

A → B | B + A +) * precedence
B → C | B * C * right assoc.
C → a | b | c + left assoc.

Recursive Descent Parsing

<expr> → <term> {(+|-) <term>}
<term> → <factor> {(*|/) <factor>}
<factor> → id | int_constant | (<expr>)

```
void expr()
  term();
  while (nT == ADD_OP || nT == SUB_OP){
    lex();
    term();
  }
void term()
  factor();
  while(nT == MULT_OP || nT == DIV_OP){
    lex()
    factor
  }
```

```
void factor()
  if (nT == IDENT || nT == INT_LIT)
    lex();
  else
    if (nT == LEFT_PAR)
      lex()
      expr()
      if(nT == RIGHT_PAR)
        lex()
      else
        error()
    else
      error()
```

Prolog

```
s(m).
s(d).
t(k).
t(i).
d(c).
d(j).
solve(3,4,5).
solve(X,Y,Z) :- s(X), !, t(Y), !, d(Z).
solve(1,2,3).
```

the query: solve(X,Y,Z)

without cut operator:

X = 3	X = d	
Y = 4	Y = k	
Z = 5	Z = c	
X = M	X = d	
Y = k	Y = k	
Z = c	Z = j	
X = M	X = d	
Y = k	Y = i	
Z = j	Z = c	
X = M	X = d	
Y = i	Y = i	
Z = c	Z = j	
X = M	X = 1	
Y = i	Y = 2	
Z = j	Z = 3	

with cut operator:

X = 3	X = M	X = M
Y = 4	Y = k	Y = k
Z = 5	Z = c	Z = j

with cut operator:

X = 3	X = M
Y = 4	Y = i
Z = 5	Z = j
X = M	
Y = k	
Z = c	
X = M	
Y = k	
Z = j	
X = M	
Y = i	
Z = c	

```
grade(a, csel200, 100).

gpa-student(X, GPA) :- grade(X, csel200, Grade1), grade(X, csel241, Grade2),
                       GPA is (Grade1 + Grade2) / 2.
```